Cryptography

Lecture 9: Number Theory and an Intro to Public-Key Cryptography

Arkady Yerukhimovich

November 9, 2026

Outline

1. A Modern Cryptography Approach
2. Modular arithmetic
3. A very brief introduction to group theory
4. Crypto Hardness Assumptions
5. Private-Key Crypto from Number Theoretic Assumptions

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

What's The Problem?

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

What's The Problem?

- All practical block-cipher constructions rely on sequence of random-looking steps (e.g., random permutation, shift bits, taking subsets of bits,...)

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

What's The Problem?

- All practical block-cipher constructions rely on sequence of random-looking steps (e.g., random permutation, shift bits, taking subsets of bits,...)
- No known security reduction to a clean (mathematical) problem

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

What's The Problem?

- All practical block-cipher constructions rely on sequence of random-looking steps (e.g., random permutation, shift bits, taking subsets of bits,...)
- No known security reduction to a clean (mathematical) problem
- Have to trust that existing cryptanalysis of block-ciphers (e.g., AES) captures all possible attacks

Let's Look At Our Assumptions

How to instantiate private-key crypto, so far:

1. Use primitives like PRGs, PRFs, PRPs to build encryption, MACs
2. Instantiate PRPs using block-ciphers

What's The Problem?

- All practical block-cipher constructions rely on sequence of random-looking steps (e.g., random permutation, shift bits, taking subsets of bits,...)
- No known security reduction to a clean (mathematical) problem
- Have to trust that existing cryptanalysis of block-ciphers (e.g., AES) captures all possible attacks

Key Question

How can we build crypto on clean mathematical foundations?

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

- Allows reducing security of primitives to solving (well studied) mathematical problem

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

- Allows reducing security of primitives to solving (well studied) mathematical problem
- Problems are (often) studied even outside of crypto

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

- Allows reducing security of primitives to solving (well studied) mathematical problem
- Problems are (often) studied even outside of crypto
- Assumptions are easy to state and understand

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

- Allows reducing security of primitives to solving (well studied) mathematical problem
- Problems are (often) studied even outside of crypto
- Assumptions are easy to state and understand
- Can be stated asymptotically, not relying on fixed input/output sizes

Building Crypto from Mathematical Assumptions

Instead, modern crypto builds cryptographic primitives from clean, mathematical problems (e.g., factoring)

- Allows reducing security of primitives to solving (well studied) mathematical problem
- Problems are (often) studied even outside of crypto
- Assumptions are easy to state and understand
- Can be stated asymptotically, not relying on fixed input/output sizes

Added Functionality

We will see how this modern crypto approach leads to the development of *public-key* cryptography.

Outline

1. A Modern Cryptography Approach
2. Modular arithmetic
3. A very brief introduction to group theory
4. Crypto Hardness Assumptions
5. Private-Key Crypto from Number Theoretic Assumptions

Modular Arithmetic

Notation: For Integers a, b, N

- $[a \bmod N]$ - a modulo N, (e.g. $[15 \bmod 7] = 1$)

Modular Arithmetic

Notation: For Integers a, b, N

- $[a \bmod N]$ - a modulo N, (e.g. $[15 \bmod 7] = 1$)
- $a \equiv b \bmod N$ if $[a \bmod N] = [b \bmod N]$
  - We say, a is *congruent* to $b \mod N$

Modular Arithmetic

Notation: For Integers a, b, N

- $[a \bmod N]$ - a modulo N, (e.g. $[15 \bmod 7] = 1$)
- $a \equiv b \bmod N$ if $[a \bmod N] = [b \bmod N]$
  - We say, a is *congruent* to $b \mod N$
- Note that:

$$a = [b \bmod N] \quad \Rightarrow \quad a \equiv b \bmod N, \text{ but}$$
$$a \equiv b \bmod N \quad \not\Rightarrow \quad a = [b \bmod N]$$

Modular Arithmetic

Notation: For Integers a, b, N

- $[a \bmod N]$ - a modulo N, (e.g. $[15 \bmod 7] = 1$)
- $a \equiv b \bmod N$ if $[a \bmod N] = [b \bmod N]$
  - We say, a is *congruent* to $b \mod N$
- Note that:

$$a = [b \bmod N] \quad \Rightarrow \quad a \equiv b \bmod N, \text{ but}$$
$$a \equiv b \bmod N \quad \not\Rightarrow \quad a = [b \bmod N]$$

Example: $8 \neq [3 \bmod 5]$, but $8 \equiv 3 \bmod 5$

Modular Arithmetic

Congruence Relation

Congruence $\mod N$ is an *equivalence relation* that obeys standard rules of arithmetic.

If $a \equiv a' \mod N$, and $b \equiv b' \mod N$ then:

- $a + b \equiv a' + b' \mod N$
- $ab \equiv a'b' \mod N$

Modular Arithmetic

Congruence Relation

Congruence $\mod N$ is an *equivalence relation* that obeys standard rules of arithmetic.

If $a \equiv a' \mod N$, and $b \equiv b' \mod N$ then:

- $a + b \equiv a' + b' \mod N$
- $ab \equiv a'b' \mod N$

Example: $[654321 \cdot 54301 \mod 100] = [21 \cdot 1 \mod 100] \equiv 21 \mod 100$

Modular Arithmetic

Congruence Relation

Congruence $\mod N$ is an *equivalence relation* that obeys standard rules of arithmetic.

If $a \equiv a' \mod N$, and $b \equiv b' \mod N$ then:

- $a + b \equiv a' + b' \mod N$
- $ab \equiv a'b' \mod N$

Example: $[654321 \cdot 54301 \mod 100] = [21 \cdot 1 \mod 100] \equiv 21 \mod 100$

But, this congruence relation does not necessarily respect division

Example: $3 \cdot 2 \equiv 6 \equiv 15 \cdot 2 \mod 24$, but $3 \not\equiv 15 \mod 24$

Modular Arithmetic

Congruence Relation

Congruence $\mod N$ is an *equivalence relation* that obeys standard rules of arithmetic.

If $a \equiv a' \mod N$, and $b \equiv b' \mod N$ then:

- $a + b \equiv a' + b' \mod N$
- $ab \equiv a'b' \mod N$

Example: $[654321 \cdot 54301 \mod 100] = [21 \cdot 1 \mod 100] \equiv 21 \mod 100$

But, this congruence relation does not necessarily respect division

Example: $3 \cdot 2 \equiv 6 \equiv 15 \cdot 2 \mod 24$, but $3 \not\equiv 15 \mod 24$

Division $\mod N$

Let $b, N \in \mathbb{Z}, b \geq 1, N > 1$, b is invertible $\mod N$ (i.e., can divide by b) if and only if b and N are relatively prime

Greatest Common Divisor and the Euclidean Algorithm

Definition

For $a, b \in \mathbb{Z}$, $gcd(a, b) = c$ s.t. c is the largest integer so that $c|a$ and $c|b$

Important: a and b are relatively prime if $gcd(a, b) = 1$

Greatest Common Divisor and the Euclidean Algorithm

Definition

For $a, b \in \mathbb{Z}$, $gcd(a, b) = c$ s.t. c is the largest integer so that $c|a$ and $c|b$

Important: a and b are relatively prime if $gcd(a, b) = 1$

Euclidean Algorithm

Given Integers a, b find $c = gcd(a, b)$.

Greatest Common Divisor and the Euclidean Algorithm

Definition

For $a, b \in \mathbb{Z}$, $gcd(a, b) = c$ s.t. c is the largest integer so that $c|a$ and $c|b$

Important: a and b are relatively prime if $gcd(a, b) = 1$

Euclidean Algorithm

Given Integers a, b find $c = gcd(a, b)$.

$GCD(a, b)$:

1. If $b|a$, return b
2. Else, return $GCD(b, [a \bmod b])$

Greatest Common Divisor and the Euclidean Algorithm

Definition

For $a, b \in \mathbb{Z}$, $gcd(a, b) = c$ s.t. c is the largest integer so that $c|a$ and $c|b$

Important: a and b are relatively prime if $gcd(a, b) = 1$

Euclidean Algorithm

Given Integers a, b find $c = gcd(a, b)$.

$GCD(a, b)$:

1. If $b|a$, return b
2. Else, return $GCD(b, [a \bmod b])$

Extended Euclidean Algorithm:

- Also lets you find X, Y such that $Xa + Yb = gcd(a, b)$

Greatest Common Divisor and the Euclidean Algorithm

Definition

For $a, b \in \mathbb{Z}$, $gcd(a, b) = c$ s.t. c is the largest integer so that $c|a$ and $c|b$

Important: a and b are relatively prime if $gcd(a, b) = 1$

Euclidean Algorithm

Given Integers a, b find $c = gcd(a, b)$.

$GCD(a, b)$:

1. If $b|a$, return b
2. Else, return $GCD(b, [a \bmod b])$

Extended Euclidean Algorithm:

- Also lets you find X, Y such that $Xa + Yb = gcd(a, b)$

Finding Multiplicative Inverse

Goal

For $b, N \in \mathbb{Z}$, find $b^{-1} \bmod N$

Finding Multiplicative Inverse

Goal

For $b, N \in \mathbb{Z}$, find $b^{-1} \bmod N$

Idea: Use extended Euclidean algorithm

- Find $X, Y \in \mathbb{Z}$ s.t. $Xb + YN = gcd(b, N)$

Finding Multiplicative Inverse

Goal

For $b, N \in \mathbb{Z}$, find $b^{-1} \bmod N$

Idea: Use extended Euclidean algorithm

- Find $X, Y \in \mathbb{Z}$ s.t. $Xb + YN = gcd(b, N)$
- Recall that for b^{-1} to exist, $gcd(b, N) = 1$

Finding Multiplicative Inverse

Goal

For $b, N \in \mathbb{Z}$, find $b^{-1} \bmod N$

Idea: Use extended Euclidean algorithm

- Find $X, Y \in \mathbb{Z}$ s.t. $Xb + YN = gcd(b, N)$
- Recall that for b^{-1} to exist, $gcd(b, N) = 1$
- $Xb + YN = 1 \implies Xb = 1 - YN \implies Xb = 1 \bmod N$

Finding Multiplicative Inverse

Goal

For $b, N \in \mathbb{Z}$, find $b^{-1} \bmod N$

Idea: Use extended Euclidean algorithm

- Find $X, Y \in \mathbb{Z}$ s.t. $Xb + YN = gcd(b, N)$
- Recall that for b^{-1} to exist, $gcd(b, N) = 1$
- $Xb + YN = 1 \implies Xb = 1 - YN \implies Xb = 1 \bmod N$
- $X = b^{-1} \bmod N$

Outline

1. A Modern Cryptography Approach
2. Modular arithmetic
3. A very brief introduction to group theory
4. Crypto Hardness Assumptions
5. Private-Key Crypto from Number Theoretic Assumptions

What Is a Group?

What Is a Group?

A set G with an operation $\cdot$ satisfying:

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$
- Exponentiation in G: $g^x = g \cdot g \cdots g$ (x times)

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$
- Exponentiation in G: $g^x = g \cdot g \cdots g$ (x times)

Examples:

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$
- Exponentiation in G: $g^x = g \cdot g \cdots g$ (x times)

Examples:

- The integers, $\mathbb{Z}$, form an abelian group under addition

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$
- Exponentiation in G: $g^x = g \cdot g \cdots g$ (x times)

Examples:

- The integers, $\mathbb{Z}$, form an abelian group under addition
- The integers, $\mathbb{Z}$, are not a group under multiplication (no inverses)

What Is a Group?

A set G with an operation $\cdot$ satisfying:

1. Closure: $u, v \in G \Rightarrow uv \in G$.
2. Associativity: $(uv)w = u(vw)$.
3. Identity: there is 1_G with $u1_G = 1_G u = u$.
4. Inverses: every u has u^{-1} with $uu^{-1} = u^{-1}u = 1_G$.

In this class, our groups are also commutative/abelian (i.e, $uv = vu$).

Additional definitions:

- $|G|$ - *order* of G (number of elements in G) - For us $|G| < \infty$
- Exponentiation in G: $g^x = g \cdot g \cdots g$ (x times)

Examples:

- The integers, $\mathbb{Z}$, form an abelian group under addition
- The integers, $\mathbb{Z}$, are not a group under multiplication (no inverses)
- $\mathbb{Z}_N = \{0, 1, \ldots, N-1\}$ is a group under addition mod N

A Familiar Primitive: A Group One-Time Pad

For a finite group G, choose a uniform secret $K \leftarrow G$ and encrypt $M \in G$ by

$$C = KM, \qquad M = K^{-1}C.$$

A Familiar Primitive: A Group One-Time Pad

For a finite group G, choose a uniform secret $K \leftarrow G$ and encrypt $M \in G$ by

$$C = KM, \qquad M = K^{-1}C.$$

- For fixed M, multiplication by M is a bijection.

A Familiar Primitive: A Group One-Time Pad

For a finite group G, choose a uniform secret $K \leftarrow G$ and encrypt $M \in G$ by

$$C = KM, \qquad M = K^{-1}C.$$

- For fixed M, multiplication by M is a bijection.
- For every ciphertext C, exactly one key $K = CM^{-1}$ produces it.

A Familiar Primitive: A Group One-Time Pad

For a finite group G, choose a uniform secret $K \leftarrow G$ and encrypt $M \in G$ by

$$C = KM, \qquad M = K^{-1}C.$$

- For fixed M, multiplication by M is a bijection.
- For every ciphertext C, exactly one key $K = CM^{-1}$ produces it.
- Hence C is uniform for every M: perfect secrecy for one use.

A Familiar Primitive: A Group One-Time Pad

For a finite group G, choose a uniform secret $K \leftarrow G$ and encrypt $M \in G$ by

$$C = KM, \qquad M = K^{-1}C.$$

- For fixed M, multiplication by M is a bijection.
- For every ciphertext C, exactly one key $K = CM^{-1}$ produces it.
- Hence C is uniform for every M: perfect secrecy for one use.

Concrete example

Addition modulo 11: $M = 7$, $K = 8$, $C = 4$. Recover $M = C - K = 7 \pmod{11}$.

This generalizes the XOR one-time pad

Exponentiation Without Exponentially Many Steps

To compute g^a, repeatedly square and multiply using the binary expansion of a.

$$2^2 = 4, \quad 2^4 = 16, \quad 2^8 = 3 \pmod{23}.$$

For example, $2^{13} = 2^8 \cdot 2^4 \cdot 2 = 3 \cdot 16 \cdot 2 = 4 \pmod{23}$.

Exponentiation Without Exponentially Many Steps

To compute g^a, repeatedly square and multiply using the binary expansion of a.

$$2^2 = 4, \quad 2^4 = 16, \quad 2^8 = 3 \pmod{23}.$$

For example, $2^{13} = 2^8 \cdot 2^4 \cdot 2 = 3 \cdot 16 \cdot 2 = 4 \pmod{23}$.

- Only $O(\log a)$ multiplications are required.

Exponentiation Without Exponentially Many Steps

To compute g^a, repeatedly square and multiply using the binary expansion of a.

$$2^2 = 4, \quad 2^4 = 16, \quad 2^8 = 3 \pmod{23}.$$

For example, $2^{13} = 2^8 \cdot 2^4 \cdot 2 = 3 \cdot 16 \cdot 2 = 4 \pmod{23}$.

- Only $O(\log a)$ multiplications are required.
- We never need to construct the enormous integer g^a.

Exponentiation Without Exponentially Many Steps

To compute g^a, repeatedly square and multiply using the binary expansion of a.

$$2^2 = 4, \quad 2^4 = 16, \quad 2^8 = 3 \pmod{23}.$$

For example, $2^{13} = 2^8 \cdot 2^4 \cdot 2 = 3 \cdot 16 \cdot 2 = 4 \pmod{23}$.

- Only $O(\log a)$ multiplications are required.
- We never need to construct the enormous integer g^a.
- Efficient forward computation does not imply efficient recovery of a.

Cyclic Groups and Generators

A group is cyclic if some g generates every element:

$$G = \langle g \rangle = \{1_G, g, g^2, \ldots, g^{q-1}\}, \qquad |G| = q.$$

Cyclic Groups and Generators

A group is cyclic if some g generates every element:

$$G = \langle g \rangle = \{1_G, g, g^2, \ldots, g^{q-1}\}, \qquad |G| = q.$$

- The order of g is the smallest positive q with $g^q = 1_G$.

Cyclic Groups and Generators

A group is cyclic if some g generates every element:

$$G = \langle g \rangle = \{1_G, g, g^2, \ldots, g^{q-1}\}, \qquad |G| = q.$$

- The order of g is the smallest positive q with $g^q = 1_G$.
- When g generates G, each group element has a unique exponent in $\mathbb{Z}_q$.

Cyclic Groups and Generators

A group is cyclic if some g generates every element:

$$G = \langle g \rangle = \{1_G, g, g^2, \ldots, g^{q-1}\}, \qquad |G| = q.$$

- The order of g is the smallest positive q with $g^q = 1_G$.
- When g generates G, each group element has a unique exponent in $\mathbb{Z}_q$.
- $g^a g^b = g^{a+b}$ and $(g^a)^b = g^{ab}$, with exponents modulo q.

Cyclic Groups and Generators

A group is cyclic if some g generates every element:

$$G = \langle g \rangle = \{1_G, g, g^2, \ldots, g^{q-1}\}, \qquad |G| = q.$$

- The order of g is the smallest positive q with $g^q = 1_G$.
- When g generates G, each group element has a unique exponent in $\mathbb{Z}_q$.
- $g^a g^b = g^{a+b}$ and $(g^a)^b = g^{ab}$, with exponents modulo q.
- A uniform exponent in $\mathbb{Z}_q$ gives a uniform element of G.

Our Running Example: A Prime-Order Subgroup

Take multiplication modulo $p = 23$ and $g = 2$.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

- $2^{11} = 1 \pmod{23}$, so these elements form a group of order $q = 11$.
- Group operations reduce modulo 23.
- Exponents reduce modulo 11.

Two Moduli, Two Different Jobs

Object	Example	Arithmetic
Group element	$g^a = 8 \in G$	multiply modulo $p = 23$

Two Moduli, Two Different Jobs

Object	Example	Arithmetic
Group element	$g^a = 8 \in G$	multiply modulo $p = 23$
Exponent / scalar	$a = 3 \in \mathbb{Z}_{11}$	add/multiply modulo $q = 11$

Two Moduli, Two Different Jobs

Object	Example	Arithmetic
Group element	$g^a = 8 \in G$	multiply modulo $p = 23$
Exponent / scalar	$a = 3 \in \mathbb{Z}_{11}$	add/multiply modulo $q = 11$

$$g^8 g^7 = g^{15} = g^4, \qquad 3 \cdot 13 = 16 \pmod{23}.$$

Exercise: Get Comfortable with the Group

Use $p = 23$, $q = 11$, $g = 2$ and the power table.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

1. Compute $g^7 g^8$ and identify its exponent modulo q.

Exercise: Get Comfortable with the Group

Use $p = 23$, $q = 11$, $g = 2$ and the power table.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

1. Compute $g^7 g^8$ and identify its exponent modulo q.
2. Compute $(g^3)^4$.

Exercise: Get Comfortable with the Group

Use $p = 23$, $q = 11$, $g = 2$ and the power table.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

1. Compute $g^7 g^8$ and identify its exponent modulo q.
2. Compute $(g^3)^4$.
3. Find the group inverse of $g^3 = 8$.

Exercise: Get Comfortable with the Group

Use $p = 23$, $q = 11$, $g = 2$ and the power table.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

1. Compute $g^7 g^8$ and identify its exponent modulo q.
2. Compute $(g^3)^4$.
3. Find the group inverse of $g^3 = 8$.
4. Solve $3x = 7 \pmod{q}$.

Exercise: Get Comfortable with the Group

Use $p = 23$, $q = 11$, $g = 2$ and the power table.

a	0	1	2	3	4	5	6	7	8	9	10
$2^a \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

1. Compute $g^7 g^8$ and identify its exponent modulo q.
2. Compute $(g^3)^4$.
3. Find the group inverse of $g^3 = 8$.
4. Solve $3x = 7 \pmod{q}$.
5. Why does our group have 11 elements rather than 22?

Group Theory Summary

- This is by no means a complete intro to group theory
- We only define what we need
- Will focus on finite, cyclic groups of prime order
- This will be enough for the primitives we cover in class

Outline

1. A Modern Cryptography Approach
2. Modular arithmetic
3. A very brief introduction to group theory
4. Crypto Hardness Assumptions
5. Private-Key Crypto from Number Theoretic Assumptions

What Are Hardness Assumptions?

- As we've discussed before, all "practical" crypto primitives rely on computational hardness
- Thus, we need to assume that some problem is hard to compute
- We have seen such assumptions before: E.g., Existence of PRG, PRF, CRHF

What Are Hardness Assumptions?

- As we've discussed before, all "practical" crypto primitives rely on computational hardness
- Thus, we need to assume that some problem is hard to compute
- We have seen such assumptions before: E.g., Existence of PRG, PRF, CRHF
- Going forward, we will instead use hard problems from number theory and mathematics
  - Some of these math concepts have been studied for 1000s of years
  - Easy to state and widely understood
  - Still believed to be hard for all PPT machines

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

Gen(1^n):

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

Gen(1^n):

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{DLog}_{\mathcal{A},\text{Gen}}(n)$

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

$\text{Gen}(1^n)$:

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{DLog}_{\mathcal{A},\text{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \text{Gen}(1^n)$, $h \leftarrow G$, sends (G, q, g, h) to $\mathcal{A}$

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

$\text{Gen}(1^n)$:

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{DLog}_{\mathcal{A},\text{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \text{Gen}(1^n)$, $h \leftarrow G$, sends (G, q, g, h) to $\mathcal{A}$
- $\mathcal{A}$ outputs $x \in \mathbb{Z}_q$

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

$\text{Gen}(1^n)$:

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{DLog}_{\mathcal{A},\text{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \text{Gen}(1^n)$, $h \leftarrow G$, sends (G, q, g, h) to $\mathcal{A}$
- $\mathcal{A}$ outputs $x \in \mathbb{Z}_q$
- We say that $\text{DLog}_{\mathcal{A},\text{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $g^x = h$.

Discrete Log Assumption

Let G be a cyclic group of order q with generator g

Discrete Log Problem

Given $h \in G$, find $0 \leq x \leq q-1$ s.t. $g^x = h$. We say $x = \log_g h$

$\text{Gen}(1^n)$:

- Pick n-bit prime q, group G of order q, generator g. Output (G, q, g)

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{DLog}_{\mathcal{A},\text{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \text{Gen}(1^n)$, $h \leftarrow G$, sends (G, q, g, h) to $\mathcal{A}$
- $\mathcal{A}$ outputs $x \in \mathbb{Z}_q$
- We say that $\text{DLog}_{\mathcal{A},\text{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $g^x = h$.

Definition: DLog is hard relative to Gen if for all PPT $\mathcal{A}$ it holds that

$$\Pr[\text{DLog}_{\mathcal{A},\text{Gen}}(n) = 1] \leq \text{negl}(n)$$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Computation Diffie-Hellman: $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n)$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Computation Diffie-Hellman: $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, sends $(G, q, g, h_1 = g^x, h_2 = g^y)$ to $\mathcal{A}$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Computation Diffie-Hellman: $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, sends $(G, q, g, h_1 = g^x, h_2 = g^y)$ to $\mathcal{A}$
- $\mathcal{A}$ outputs $h_3 \in G$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Computation Diffie-Hellman: $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, sends $(G, q, g, h_1 = g^x, h_2 = g^y)$ to $\mathcal{A}$
- $\mathcal{A}$ outputs $h_3 \in G$
- We say that $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $h_3 = g^{xy}$.

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 1: Computational Diffie-Hellman

- Hard to find g^{xy}

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Computation Diffie-Hellman: $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, sends $(G, q, g, h_1 = g^x, h_2 = g^y)$ to $\mathcal{A}$
- $\mathcal{A}$ outputs $h_3 \in G$
- We say that $\mathrm{CDH}_{\mathcal{A},\mathrm{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $h_3 = g^{xy}$.

Definition: CDH is hard if for all PPT $\mathcal{A}$: $\Pr[\mathcal{A} \text{ wins}] \leq \mathrm{negl}(n)$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, $b \leftarrow \{0, 1\}$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, $b \leftarrow \{0, 1\}$
- If $b = 0$, send $(G, q, g, g^x, g^y, g^{xy})$ to $\mathcal{A}$.
  If $b = 1$, choose $z \leftarrow \mathbb{Z}_q$, and send (G, q, g, g^x, g^y, g^z) to $\mathcal{A}$

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, $b \leftarrow \{0, 1\}$
- If $b = 0$, send $(G, q, g, g^x, g^y, g^{xy})$ to $\mathcal{A}$.
  If $b = 1$, choose $z \leftarrow \mathbb{Z}_q$, and send (G, q, g, g^x, g^y, g^z) to $\mathcal{A}$
- $\mathcal{A}$ outputs bit b'

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, $b \leftarrow \{0, 1\}$
- If $b = 0$, send $(G, q, g, g^x, g^y, g^{xy})$ to $\mathcal{A}$.
  If $b = 1$, choose $z \leftarrow \mathbb{Z}_q$, and send (G, q, g, g^x, g^y, g^z) to $\mathcal{A}$
- $\mathcal{A}$ outputs bit b'
- We say that $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $b = b'$.

Diffie-Hellman Assumptions

Let G be a cyclic group of order q with generator g

Diffie-Hellman Problem

Given $h_1 = g^x$, $h_2 = g^y$, find g^{xy}

Variant 2: Decisional Diffie-Hellman

- Hard to distinguish g^{xy} from a random element in G

Consider the following game between an adversary $\mathcal{A}$ and a challenger:

Decisional Diffie-Hellman: $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n)$

- Challenger runs $(G, q, g) \leftarrow \mathrm{Gen}(1^n)$, $x, y \leftarrow \mathbb{Z}_q$, $b \leftarrow \{0, 1\}$
- If $b = 0$, send $(G, q, g, g^x, g^y, g^{xy})$ to $\mathcal{A}$.
  If $b = 1$, choose $z \leftarrow \mathbb{Z}_q$, and send (G, q, g, g^x, g^y, g^z) to $\mathcal{A}$
- $\mathcal{A}$ outputs bit b'
- We say that $\mathrm{DDH}_{\mathcal{A},\mathrm{Gen}}(n) = 1$ (i.e., $\mathcal{A}$ wins) if $b = b'$.

Definition: DDH is hard if for all PPT $\mathcal{A}$: $\Pr[\mathcal{A} \text{ wins}] \leq 1/2 + \mathrm{negl}(n)$

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

1. DLog vs. CDH

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

1. DLog vs. CDH
   - If $\mathcal{A}$ can solve DLog, then given g^x, g^y, he can find x, y and compute g^{xy} (solve CDH)
   - The reverse direction doesn't seem true

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

1. DLog vs. CDH
   - If $\mathcal{A}$ can solve DLog, then given g^x, g^y, he can find x, y and compute g^{xy} (solve CDH)
   - The reverse direction doesn't seem true
2. CDH vs. DDH

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

1. DLog vs. CDH
   - If $\mathcal{A}$ can solve DLog, then given g^x, g^y, he can find x, y and compute g^{xy} (solve CDH)
   - The reverse direction doesn't seem true
2. CDH vs. DDH
   - If $\mathcal{A}$ given g^x, g^y can find g^{xy}, then he can distinguish this from g^z for a random $z \leftarrow \mathbb{Z}_q$ (solve DDH)
   - The reverse direction doesn't seem true

Relationship Between Cyclic Group Assumptions

Relative Hardness of The Problems

$DLog \geq CDH \geq DDH$

1. DLog vs. CDH
   - If $\mathcal{A}$ can solve DLog, then given g^x, g^y, he can find x, y and compute g^{xy} (solve CDH)
   - The reverse direction doesn't seem true
2. CDH vs. DDH
   - If $\mathcal{A}$ given g^x, g^y can find g^{xy}, then he can distinguish this from g^z for a random $z \leftarrow \mathbb{Z}_q$ (solve DDH)
   - The reverse direction doesn't seem true

Strength of Assumption

Since DDH is the easiest problem, assuming it is secure is the strongest assumption

Preference for Prime-Order Groups

Reasons to prefer prime-order groups:

- Easy to find a generator, all $g \in G$ (except the identity) are generators.

Preference for Prime-Order Groups

Reasons to prefer prime-order groups:

- Easy to find a generator, all $g \in G$ (except the identity) are generators.
- DDH is easy in some composite-order groups!!!

Preference for Prime-Order Groups

Reasons to prefer prime-order groups:

- Easy to find a generator, all $g \in G$ (except the identity) are generators.
- DDH is easy in some composite-order groups!!!

Group of Quadratic Residues mod p

Preference for Prime-Order Groups

Reasons to prefer prime-order groups:

- Easy to find a generator, all $g \in G$ (except the identity) are generators.
- DDH is easy in some composite-order groups!!!

Group of Quadratic Residues mod p

Let $p = 2q + 1$ with both p and q prime.

Preference for Prime-Order Groups

Reasons to prefer prime-order groups:

- Easy to find a generator, all $g \in G$ (except the identity) are generators.
- DDH is easy in some composite-order groups!!!

Group of Quadratic Residues mod p

Let $p = 2q + 1$ with both p and q prime.

$$G = \{[h^2 \bmod p] | h \in \mathbb{Z}_p^*\}$$

is a group of prime order q

Note: The group we looked at before: $p = 23$, $q = 11$, $g = 2$ is such a group

$$2 \equiv 5^2 \bmod 23$$

Outline

1. A Modern Cryptography Approach
2. Modular arithmetic
3. A very brief introduction to group theory
4. Crypto Hardness Assumptions
5. Private-Key Crypto from Number Theoretic Assumptions

PRG from DDH

Let G be a group of order q with generator g

PRG from DDH

Let G be a group of order q with generator g

PRG Construction

Given $s = (x, y) \leftarrow \mathbb{Z}_q^2$, output $G(s) = (g^x, g^y, g^{xy})$

PRG from DDH

Let G be a group of order q with generator g

PRG Construction

Given $s = (x, y) \leftarrow \mathbb{Z}_q^2$, output $G(s) = (g^x, g^y, g^{xy})$

- Expansion:
  - s consists of two random integers in $\mathbb{Z}_q$
  - $G(s)$ outputs 3 group elements in G of order q
  - So $|G(s)| > |s|$

PRG from DDH

Let G be a group of order q with generator g

PRG Construction

Given $s = (x, y) \leftarrow \mathbb{Z}_q^2$, output $G(s) = (g^x, g^y, g^{xy})$

- Expansion:
  - s consists of two random integers in $\mathbb{Z}_q$
  - $G(s)$ outputs 3 group elements in G of order q
  - So $|G(s)| > |s|$
- Pseudorandomness:

PRG from DDH

Let G be a group of order q with generator g

PRG Construction

Given $s = (x, y) \leftarrow \mathbb{Z}_q^2$, output $G(s) = (g^x, g^y, g^{xy})$

- Expansion:
  - s consists of two random integers in $\mathbb{Z}_q$
  - $G(s)$ outputs 3 group elements in G of order q
  - So $|G(s)| > |s|$
- Pseudorandomness:
  - g^x, g^y are random group elements

PRG from DDH

Let G be a group of order q with generator g

PRG Construction

Given $s = (x, y) \leftarrow \mathbb{Z}_q^2$, output $G(s) = (g^x, g^y, g^{xy})$

- Expansion:
  - s consists of two random integers in $\mathbb{Z}_q$
  - $G(s)$ outputs 3 group elements in G of order q
  - So $|G(s)| > |s|$
- Pseudorandomness:
  - g^x, g^y are random group elements
  - If $\mathcal{A}_c$ can distinguish (g^x, g^y, g^{xy}) from random, then $\mathcal{A}_r$ just runs $\mathcal{A}_c$ to break DDH

Collision-Resistant Compression from DLog

Construction: $G = \langle g \rangle$ of prime order q

Choose public $h \leftarrow G$. For $(x, y) \in \mathbb{Z}_q^2$, set

$$H^h(x, y) = g^x h^y \in G.$$

Collision-Resistant Compression from DLog

Construction: $G = \langle g \rangle$ of prime order q

Choose public $h \leftarrow G$. For $(x, y) \in \mathbb{Z}_q^2$, set

$$H^h(x, y) = g^x h^y \in G.$$

Reduction: Given a DLog challenge $h = g^a$, publish h and run the collision finder.

Collision-Resistant Compression from DLog

Construction: $G = \langle g \rangle$ of prime order q

Choose public $h \leftarrow G$. For $(x, y) \in \mathbb{Z}_q^2$, set

$$H^h(x, y) = g^x h^y \in G.$$

Reduction: Given a DLog challenge $h = g^a$, publish h and run the collision finder.

1. A collision gives distinct $(x, y), (x', y')$ with

$$g^x h^y = g^{x'} h^{y'} \quad \Longrightarrow \quad x - x' \equiv a(y' - y) \pmod{q}.$$

Collision-Resistant Compression from DLog

Construction: $G = \langle g \rangle$ of prime order q

Choose public $h \leftarrow G$. For $(x, y) \in \mathbb{Z}_q^2$, set

$$H^h(x, y) = g^x h^y \in G.$$

Reduction: Given a DLog challenge $h = g^a$, publish h and run the collision finder.

1. A collision gives distinct $(x, y), (x', y')$ with
$$g^x h^y = g^{x'} h^{y'} \quad \Longrightarrow \quad x - x' \equiv a(y' - y) \pmod{q}.$$

2. If $y = y'$, then $x = x'$ in $\mathbb{Z}_q$ – not a collision. Thus $y' - y \neq 0$ and is invertible modulo prime q.

Collision-Resistant Compression from DLog

Construction: $G = \langle g \rangle$ of prime order q

Choose public $h \leftarrow G$. For $(x, y) \in \mathbb{Z}_q^2$, set

$$H^h(x, y) = g^x h^y \in G.$$

Reduction: Given a DLog challenge $h = g^a$, publish h and run the collision finder.

1. A collision gives distinct $(x, y), (x', y')$ with
$$g^x h^y = g^{x'} h^{y'} \quad \Longrightarrow \quad x - x' \equiv a(y' - y) \pmod{q}.$$
2. If $y = y'$, then $x = x'$ in $\mathbb{Z}_q$ – not a collision. Thus $y' - y \neq 0$ and is invertible modulo prime q.
3. Recover the discrete logarithm:
$$\boxed{a = (x - x')(y' - y)^{-1} \bmod q.}$$

PRF from DDH (Naor-Reingold PRF)

Let G be a group of order q with generator g

PRF from DDH (Naor-Reingold PRF)

Let G be a group of order q with generator g

PRF Construction

Let $k = (a_0, a_1, \ldots, a_n) \leftarrow \mathbb{Z}_q^{n+1}$. On input $x \in \{0,1\}^n$

$$F_k(x) = g^{a_0 \prod_{i=1}^{n} a_i^{x_i}} \qquad (x_i = i^{th} \text{ bit of } x)$$

PRF from DDH (Naor-Reingold PRF)

Let G be a group of order q with generator g

PRF Construction

Let $k = (a_0, a_1, \ldots, a_n) \leftarrow \mathbb{Z}_q^{n+1}$. On input $x \in \{0,1\}^n$

$$F_k(x) = g^{a_0 \prod_{i=1}^{n} a_i^{x_i}} \qquad (x_i = i^{th} \text{ bit of } x)$$

Pseudorandomness (Intuition):

PRF from DDH (Naor-Reingold PRF)

Let G be a group of order q with generator g

PRF Construction

Let $k = (a_0, a_1, \ldots, a_n) \leftarrow \mathbb{Z}_q^{n+1}$. On input $x \in \{0,1\}^n$

$$F_k(x) = g^{a_0 \prod_{i=1}^{n} a_i^{x_i}} \qquad (x_i = i^{th} \text{ bit of } x)$$

Pseudorandomness (Intuition):

- For every new input x', $F_k(x')$ differs from $F_k(x)$ by at least one term in the exponent

PRF from DDH (Naor-Reingold PRF)

Let G be a group of order q with generator g

PRF Construction

Let $k = (a_0, a_1, \ldots, a_n) \leftarrow \mathbb{Z}_q^{n+1}$. On input $x \in \{0,1\}^n$

$$F_k(x) = g^{a_0 \prod_{i=1}^{n} a_i^{x_i}} \qquad (x_i = i^{th} \text{ bit of } x)$$

Pseudorandomness (Intuition):

- For every new input x', $F_k(x')$ differs from $F_k(x)$ by at least one term in the exponent
- By DDH and hybrid arguments, we cannot distinguish the resulting outputs from random